

22072045 AlishBan (1)

 Islington College,Nepal

Document Details

Submission ID

trn:oid:::3618:79394522

Submission Date

Jan 17, 2025, 7:01 AM GMT+5:45

Download Date

Jan 17, 2025, 7:02 AM GMT+5:45

File Name

22072045 AlishBan (1)

File Size

38.3 KB

50 Pages

5,610 Words

33,195 Characters

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **46 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 2%  Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 46 Not Cited or Quoted 8%**
Matches with neither in-text citation nor quotation marks
- 3 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3% Internet sources
- 2% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Informatics Education Limited on 2011-08-30	1%
2	Submitted works	Central Queensland University on 2023-04-07	<1%
3	Submitted works	Academic City University on 2024-05-27	<1%
4	Internet	ironpdf.com	<1%
5	Submitted works	Washington University of Science and Technology on 2023-08-23	<1%
6	Submitted works	University of Greenwich on 2024-05-15	<1%
7	Submitted works	ACS Egham International School on 2023-05-31	<1%
8	Submitted works	Manav Rachna International University on 2023-06-15	<1%
9	Submitted works	National University of Ireland, Maynooth on 2024-04-21	<1%
10	Internet	www.asap-utilities.com	<1%

11	Submitted works	Informatics Education Limited on 2009-11-10	<1%
12	Submitted works	Botswana Open University on 2023-09-17	<1%
13	Submitted works	Institute Of Business Management & Research, IPS on 2024-11-27	<1%
14	Publication	McGarry, Shane. "Expanding the Frame: Realising Engagement Through an Intera..."	<1%
15	Internet	dev.to	<1%
16	Submitted works	University of Wales Institute, Cardiff on 2022-10-20	<1%
17	Submitted works	Georgetown University on 2024-08-13	<1%
18	Submitted works	American Intercontinental University Online on 2024-11-12	<1%
19	Submitted works	Informatics Education Limited on 2011-08-29	<1%
20	Submitted works	Informatics Education Limited on 2011-08-30	<1%
21	Submitted works	South Bank University on 2017-05-17	<1%
22	Submitted works	The University of Texas at Arlington on 2024-03-02	<1%
23	Internet	cs.worcester.edu	<1%
24	Internet	dspace.univ-msila.dz	<1%

25	Internet	
ebin.pub		<1%
26	Internet	
www.coursehero.com		<1%
27	Submitted works	
Southern University And A & M College on 2024-04-30		<1%
28	Submitted works	
University of Ulster on 2024-12-11		<1%

Project Overview.

Introduction.

In today's fast-paced world, effective management of personal finances becomes crucial for individuals living in bustling urban environments. As the complexity of personal financial management increases, so does the need for efficient tools to handle this challenge. SpendWise, a desktop application developed using the .Net Framework and C# programming language, aims to address these needs by providing robust, reliable, and easy-to-use financial tracking and management tools.

SpendWise offers a comprehensive solution for budget tracking, expense management, and financial planning. It allows users to track both their cash inflows (such as salaries, bonuses, and other earnings) and outflows (including bills, groceries, and discretionary spending). The application is designed to help users maintain a balanced budget, manage debts efficiently, and plan for future financial stability.

C# and the .NET Framework have been chosen for the development of SpendWise due to the robust framework support, strong typing, object-oriented capabilities, and an extensive library of functions that facilitate rapid development and effective handling of complex financial data. Complementing this is Visual Studio 2022, an IDE that

supports the comprehensive development cycle of SpendWise, including coding, debugging, deployment, and maintenance. This IDE is equipped with rich features

such as integrated debugging tools, a wide range of extensions and plugins, and support for multiple programming languages, ensuring a smooth and efficient development process.

The architecture of SpendWise leverages the .Net Framework's capabilities, including the Common Language Runtime (CLR) for managing execution and the extensive Class Library for accessing essential functionalities such as database connections, file handling, and more. This ensures that SpendWise not only meets the current financial management needs but also remains scalable and adaptable to future enhancements.

In conclusion, SpendWise is designed to transform the way individuals manage their finances, offering a user-friendly interface combined with powerful backend technology to ensure accuracy, security, and reliability in financial management.

Scope of the project Spendwise.

"SpendWise" is designed to be a comprehensive desktop application focused on personal finance management. Developed using the .Net Framework and C#, it aims to simplify financial tracking and planning for users in bustling urban environments.

Here's a detailed scope of the project:

Development of a Desktop Application: Utilizing the .Net Framework and C#

programming language to build a user-friendly desktop application for personal finance management.

Financial Tracking and Management: Enabling users to comprehensively track and manage their cash inflows (e.g., salaries, bonuses), outflows (e.g., bills, groceries).

Budgeting Tools: Incorporating tools to assist users in creating and maintaining a balanced budget.

Debt Management: Offering functionalities to manage and plan debt repayment efficiently.

Currency Selection: Allowing users to select their preferred currency, such as USD or NPR, enhancing the application's accessibility and customization.

Reporting: Providing features for generating and analyzing financial reports to help users make informed financial decisions.

User Interface: Designing an intuitive interface that caters to a broad user base with varying financial management experience.

Security Features: Implementing robust security measures to protect user data.

Testing and Deployment: Ensuring thorough testing is carried out to guarantee functionality and reliability before deployment.

Objectives.

To ensure that "SpendWise" effectively meets the needs of its users for comprehensive financial management, the project is driven by several key objectives.

These goals focus on enhancing the overall user experience, improving financial literacy, and providing robust technical support. Here are the specific objectives for the

SpendWise project:

Enhance Financial Awareness: Provide users with tools to track and manage their personal finances, enhancing their understanding and control over their financial health.

Simplify Financial Management: Create an intuitive and easy-to-use interface that simplifies the process of managing incomes and expenses.

Promote Financial Planning: Equip users with planning tools to set and achieve financial goals, such as saving for future needs or paying off debts.

25 Improve Financial Decision-Making: Offer detailed reports and analytics that help users make informed financial decisions based on their income, spending patterns, and financial obligations.

Customize User Experience: Allow users to personalize their experience by selecting preferred currencies and tailoring the interface to their financial management needs.

17 Ensure Data Security: Implement state-of-the-art security measures to protect users' sensitive financial information from unauthorized access.

Deliver Reliable Performance: Ensure the application is robust, with minimal downtime and efficient operation across various computing environments.

Technologies used

13 A technology stack, or tech stack, is essentially the combination of tools, frameworks, libraries, programming languages, and software employed to develop and operate a project or application. It typically includes the frontend, responsible for managing the user interface (e.g., React, Angular); the backend, which handles server-side logic (e.g., Node.js, .NET); the database, used for data storage and management (e.g., MySQL, MongoDB); and development and deployment tools, such as Git and Docker,

to streamline the workflow effectively (Scale, 2024). In essence, a tech stack serves as the foundation or "ingredients" required to create and maintain a software application (Nicole Abramowski, 2023).

Technology stack are Given below:

Framework: MAUI Blazor Hybrid.

MAUI Blazor Hybrid is a versatile framework that integrates .NET MAUI (Multi-platform App UI) with Blazor to create cross-platform applications. It enables developers to utilize Blazor components and Razor syntax for designing user interfaces while harnessing .NET MAUI's access to native device functionalities. This hybrid approach facilitates the development of web-based user interfaces with native application capabilities for platforms like Android, iOS, Windows, and macOS, all from a single codebase. (Rastegarinia, 2023).

Figure 1 MAUI Blazor Hybrid Logo (Rastegarinia, 2023)

External Libraries: Newtonsoft.Json.

Newtonsoft.Json, also known as Json.NET, is a widely used open-source .NET library for parsing, serializing, and deserializing JSON data. It simplifies the conversion of .NET objects to JSON format and vice versa, offering fast and flexible JSON handling. Commonly utilized in web APIs, it provides features like LINQ to JSON, custom converters, and high performance, making it a staple in .NET applications for managing JSON data effectively.

(Json.NET Documentation, 2024).

Persistence Mechanism: File handling using JSON.

File handling with JSON entails reading and writing JSON files to store or retrieve application data. In .NET, libraries such as Newtonsoft.Json or System.Text.Json are commonly employed for serializing objects (converting them into JSON format) and deserializing JSON data (converting it back into objects). JSON files are lightweight, human-readable, and extensively utilized in modern applications for configurations, data storage, and data exchange. (Panagopoulos, 2024).

System requirements and dependencies (software/hardware)

System Requirements

Software Requirements: .NET SDK version 9.0

Hardware Requirements: Minimum of 4GB RAM and 1GB storage space.

Project Dependencies

Microsoft.AspNetCore.Components.

Microsoft.AspNetCore.Components is a framework part of the ASP.NET Core library that enables developers to build interactive user interfaces using C#. It is primarily used for developing web applications following the Blazor technology, which allows for

both client-side and server-side programming. This framework helps in creating rich interactive UIs with C# instead of JavaScript.

IronPDF (2023 8.6)

 IronPDF is a comprehensive library for the .NET framework that enables developers to create, read, and edit PDF documents programmatically. It is a versatile tool that can convert HTML to PDF, which is particularly useful for generating PDFs from web pages or HTML templates.

Installation and Setup Guide.

Installation and setup

Figure 2 Creating new project

Figure 3 Selecting project templates

Figure 4 Setting project name SpendWise

Figure 5 Selecting framework

Figure 6 Project successfully created

Figure 7 Creating Repository

Figure 8 Signing in

Figure 9 Connecting visual studio folder with github

Figure 10 Connected successfully

Figure 11 Github profile after creation

Figure 12 Github repository

Database setup (Using JSON file structure).

Figure 13 Database Setup and Initialization

In the above figure the initial setup and configuration process of the database used in the project. It includes steps such as defining the schema, establishing connections, and setting up initial parameters to ensure the database is ready for data transactions.

Figure 14 Data Loading Method.

This figure demonstrates the method used for loading data into the application from the database. It details the queries, data handling techniques, and the interaction between the application and the database to fetch and display the necessary information efficiently.

Figure 15 Data Saving Method.

 This figure shows the method employed to save data back to the database from the application. It outlines the process of gathering input from the user interface, processing the data as required, and updating the database to reflect changes made within the application.

User Guide (Testing).

Login and Account Setup.

Register Testing.

Objectives

To register the User in the platform “SpendWise”.

Actions

- i.) Run the program.
- ii.) Provide a username, email and password.

Expected Outcomes

The registration credentials should be securely stored in JSON format and login, the dashboard should be promptly displayed.

Actual Outcomes

The registration credentials was securely stored in JSON format and login, the dashboard was displayed.

Result



Test Successful.

Table 1 Test Case 1

Figure 16 Inserting User Credentials

Figure 17 Login Page.

Figure 18 Registered data in the JSON File.

Login Testing.

Objectives

To register the User in the platform “SpendWise”.

Actions

- i.) Run the program.
- ii.) Provide a username, email and Currency Type.

Expected Outcomes

The User dashboard should be displayed with currency NPR.

Actual Outcomes

The User dashboard was displayed with currency NPR.

Result

Test Successful.

Table 2 Test Case 2

Figure 19 inserting the login credentials.

Figure 20 Dashboard

Currencies Handling

Objectives

To choose the currency in the platform "SpendWise".

Actions

- i.) Run the program.
- ii.) Provide a username, email and Currency Type.

Expected Outcomes

The User dashboard should be displayed with currency of your choice.

Actual Outcomes

The User dashboard was displayed with currency of the choice.

Result

Test Successful.

Table 3 Test Case 3

Figure 21 Selecting NPR currency

Figure 22 Login from the NPR Currency.

Figure 23 Selecting USD Currency

Figure 24 Login in after Selecting USD Currency

Managing Transactions.

Adding cash inflows.

Objectives

To successfully record a cash inflow in the "SpendWise" platform.

Actions

i.) Access the Cash Inflow section.

ii.) Input the amount, select a tag from the dropdown, and enter notes in the respective fields.

iii.) Click on the 'Add Inflow' button.

Expected Outcomes

The cash inflow is recorded successfully, and the new inflow appears listed under "Cash Inflows" with the correct amount, selected tag, and notes.

Actual Outcomes

The user dashboard updates to display the newly added cash inflow with the specified details.

Result

Test Successful.

Table 4 Test Case 4

Figure 25 inserting the data for cash inflow.

Figure 26 Inflow transaction

Figure 27 Dashboard after Cash inflow.

Figure 28 Cash inflow Transaction.

Adding cash outflows.

Objectives

To successfully record a cash outflow in the "SpendWise" platform.

Actions

- i.) Access the Cash outflow section.
- ii.) Input the amount, select a tag from the dropdown, and enter notes in the respective fields.
- iii.) Click on the 'Add outflow' button.

Expected Outcomes

The cash outflow is recorded successfully, and the new inflow appears listed under "Cash Outflows" with the correct amount, selected tag, and notes.

Actual Outcomes

The user dashboard updates to display the newly added cash outflow with the specified details.

Result

Test Successful.

Table 5 Test Case 5



Figure 29 inserting the data for cash ioutflow.

Figure 30 Outflow transaction

Figure 31 Dashboard after the cash out flow

Figure 32 Cash outflow Transaction.

Adding and tracking debts.

Objectives

To successfully record a debt transaction in the "SpendWise" platform.

Actions

- i.) Access the Debt Management section.
- ii.) Enter the amount of the debt, select the creditor from the dropdown, specify the due date, and enter any relevant notes.
- iii.) Click on the 'Add Debt' button.

Expected Outcomes

The debt is recorded successfully, and the new debt transaction appears listed under "Debt Management" with the correct amount, creditor, due date, and notes.

Actual Outcomes

The user dashboard updates to display the newly recorded debt with the specified details.

Result

Test Successful.

Table 6 Test Case 6

Figure 33 Time to take the debt

Figure 34 inserting the credentials for taking debt.

Figure 35 Debt taken successfully.

Figure 36 Data of taking the debt

Adding custom notes and categories to transactions

Objectives

To successfully add custom notes and categories to a transaction in the "SpendWise" platform.

Actions

- i.) Navigate to the cashinflow section of the application.
- ii.) create a new transaction
- iii.) Enter custom notes in the provided notes field.
- iv.) Select or add a new category from the category dropdown menu.

v.) Click on the 'Add Transaction' button, depending on the action.

Expected Outcomes

The transaction should be updated or added with the custom notes and selected category correctly displayed in the transaction details.

Actual Outcomes

The transaction details on the cashinflow or in the transaction list are updated to include the new notes and category.

Result

Test Successful.

Table 7 Test Case 7

Figure 37 inserting the custom notes and categories.

Figure 38 Transaction Successful with Custom tag and categories.

Figure 39 Data of using the custom tag and categories.

Tagging transactions

Objectives

To filter the transaction from the tags

Actions

- i.) Navigate to the transaction section of the application.
- ii.) Enter the tag
- iii.) Click on the 'Filter' button.

Expected Outcomes

Only transactions tagged with the specified tag remain visible in the transaction list, effectively isolating them from other entries.

Actual Outcomes

After applying the tag filter, the transactions list updates to display only those entries that contain the selected tag, confirming the filter's accuracy and effectiveness.

Result

Test Successful.

Table 8 Test Case 8

Figure 40 Transaction before the tag filtration

Figure 41 Transaction after the tag "Party" Filtration.,

Dashboard Features

Viewing statistics (total inflows, outflows, debts, cleared debts)

Objectives

View statistics (total inflows, outflows, debts, cleared debts)

Actions

i.) Navigate to the dashboard of the application.

Expected Outcomes

The total inflows, outflows, debts, cleared debts should get visible along with that the highest and lowest of the inflows, outflows, debts are also get visible.

Actual Outcomes

The total inflows, outflows, debts, cleared debts should get visible along with that the highest and lowest of the inflows, outflows, debts was also visible

Result

Test Successful.

Table 9 Test Case 9

Figure 42 Dashboard displaying the total inflows, outflows, debts, cleared debts.

Figure 43 Total transactions data for POW.

Sorting and Filtering Transactions by Criteria in SpendWise.

Objectives

To ensure that transactions can be accurately sorted and filtered by specified criteria such as tags and transaction types in the "SpendWise" platform.

Actions

Navigate to the Transactions section of the application.

Select the sorting criteria from the dropdown menu (e.g., transaction type).

Execute the sorting and filtering by clicking on the 'Filters' button.

Expected Outcomes

Transactions should be sorted and displayed according to the selected criteria, with only transactions matching the specified tags and types visible.

Actual Outcomes

The transaction list updates to reflect the sorting order and filtered results based on the applied criteria, showcasing only those entries that meet the set conditions.

Result

Test Successful.

Table 10 Test Case 10

Figure 44 Transaction data's

Figure 45 Result after the filtration.

View Pending Debts.

Objectives

To verify the functionality of viewing pending debts and filtering these debts by a specific date range in the "SpendWise" platform.

Actions

Access the Dashboard for looking the pending debts.

Access the filter option and select or enter a specific date range for the pending debts.

Click on the 'Filter' button to update the view based on the selected date range.

Expected Outcomes

The debts list should update to only show pending debts that fall within the specified date range, effectively isolating these transactions.

Actual Outcomes

After applying the date range filter, the debts display is limited to those entries within the selected dates, confirming the filter's functionality.

 Result

Test Successful.

Table 11 Test Case 11

Figure 46 inserting the start and the end date

Figure 47 Pending Debts date in that date.

Search and Filtering.

Objectives

To verify the functionality of searching and filtering transactions by tags, dates, and amounts in the SpendWise platform.

Actions

Navigate to the Transactions section of the application.

Enter a specific tag, date range, in filter interface.

From that result search the data through notes.

Expected Outcomes

14 The transactions list should update to only show transactions that match the specified search criteria.

Actual Outcomes

14 The transactions list was update to only show transactions that match the specified search criteria.

Result

Test Successful.

Table 12 Test Case 12

Figure 48 Transaction before filtering and search

Figure 49 after filter before searching.

Figure 50 Filter result searching.

Import and Export.

Steps for importing data

Objectives

To verify the functionality of importing transaction data into the "SpendWise" platform and ensuring the data reflects accurately in the system.

Actions

Navigate to the import data interface on the transaction.

Click on the 'Choose File' button to select the appropriate data file (e.g., CSV or Excel)

that contains transaction data.

Click on the 'Import' button to begin the data import process.

Expected Outcomes

The transaction data from the file should be successfully imported into the SpendWise platform

Actual Outcomes

21 The transaction data from the file was successfully imported into the SpendWise platform

26 Result

Test Successful.

Table 13 Test Case 13

Figure 51 Before Importing the Package json File.

Figure 52 After importing the Package Json.

Steps for exporting data.

Objectives

To verify the functionality of exporting transaction data from the "SpendWise" platform.

Actions

Navigate to the transaction.

Click on the 'Download as PDF' button to download the transaction data as pdf.

Expected Outcomes

The transaction data from the file should be successfully exported from the SpendWise platform

Actual Outcomes

The transaction data from the file was successfully exported from the SpendWise platform

Result

Test Successful.

Table 14 Test Case 14

Figure 53 clicking the download as pdf button.

Figure 54 Downloaded notification

Figure 55 Recent exported transaction.

Handling Errors and Troubleshooting.

Syntax Error.

Import Failure

The system fails to complete the data import process

Actions

Verify the file format is supported.

Check the file integrity and attempt re-import.

Ensure the system has the correct permissions to access and modify the data

Expected Outcomes

Increase system resources if necessary

Optimize the import process for better performance

Solution

The code starts working

Result

Error Handled.

Table 15 Test Case 15

Figure 56 Error due to fail import.

Syntax Error.

Obsolete Method Warning (CS0618)

"HtmlToPdf" is marked obsolete, with a suggestion to use "ChromePdfRenderer"

Actions

1. Review documentation for the new ChromePdfRenderer.
2. Refactor the existing code to implement the new method.
3. Test the application to ensure the new method works as expected.

Expected Outcomes

1. The DLL file is successfully copied without any access restrictions.
2. The build process completes without errors.

Solution

Resolve file locking issues by ensuring exclusive access during the build.

Result

Build Successful and Errors Resolved.

Table 16 Test Case 16

Figure 57 Error

Figure 58 successfully run.

Visit Appendix for more Testing's

System Design and Architecture.

Database Schema.

Figure 59 Database Schema of the platform.

Sample JSON Data

Figure 60 Sample of JSON Data

Software Architecture Diagram.

The software architecture diagram assists one in abstracting the overall outline of the application. The constraints, relationships, and boundaries between the elements can be figured out with the help of the software architecture diagram. Overall, it provides the physical structure of the software deployment and its evolution roadmap.

Figure 61 System Architecture Diagram.

From the software architecture diagram of the "SpendWise" desktop application, the user flow and system relationships are clearly delineated, ensuring an intuitive user experience. Upon launching the application, users are directly taken to the login page. If not previously registered, they must complete the registration process to access the system. Following a successful login, users enter the dashboard, which acts as the central hub for financial management activities including CashInflow and CashOutflow. Transactions entered on these pages are saved in a JSON format named 'data'. The CashOutflow page also features a conditional 'Take Debt' form that activates if the

outflow exceeds the user's total balance. Furthermore, users can navigate to a transaction overview page from the dashboard, where they can download transaction details in PDF format, enhancing the application's usability for comprehensive financial oversight.

Flow Charts

Figure 62 Flowchart of the platform

Milestone Chart.

A milestone chart is a type of project management tool used to visually represent significant events within a project timeline, known as milestones. These milestones are crucial points that mark major events, decision points, or the completion of key phases of the project. The primary purpose of a milestone chart is to provide a clear and simple overview of a project's progress and to highlight the important achievements along the project timeline.

Figure 63 Milestone Chart.

UML Diagrams

UML (Unified Modeling Language) diagrams are a type of visual representation used to describe, specify, design, and document the structure and behaviour of systems,

2 particularly software applications. UML is a standardized modelling language that consists of an integrated set of diagrams, developed to help system and software developers for specifying, visualizing, constructing, and documenting the artefacts of software systems, as well as for business modelling and other non-software systems.

Use Case

Figure 64 Use case of the platform.

ERD Diagrams.

Data or Entity Modeling is a crucial aspect of designing any application, including the SpendWise project. It involves structuring and defining the data objects (entities) that the application will utilize and understanding the relationships between these entities (Visual Paradigm, 2024).

Key concepts of Data/Entity Modeling are given below:

3 Entities

22 An entity refers to a real-world object, concept, or thing for which data is stored and systematically organized. In databases, an entity is typically represented as a table, where rows (or records) correspond to specific instances of the entity, and columns

3 define its attributes or properties. For instance, a "User" entity might include attributes

such as UserID, Name, Email, and Age, with each row in the table capturing details of an individual user.

(Ellis, 2023).

Figure 65 Entity name Users

Fields/Attributes

Attributes, also known as fields, are the defining properties or characteristics of an entity that outline its structure and store its data. In databases, attributes are

represented as columns in a table, where each column holds a specific type of

information about the entity. For instance, a "Product" entity might include attributes

like ProductID, Name, Price, and Category, which describe various aspects of a

product. In object-oriented programming, attributes are represented as fields or

properties within a class, encapsulating the data associated with an object.

(PHOENIXNAP, 2021).

Figure 66 Attributes name UserID

Relationships

Relationships describe the connections or interactions between entities. In databases, they establish logical associations between tables, such as one-to-one, one-to-many, or many-to-many relationships. For instance, in an e-commerce database, a "Customer" entity might have a one-to-many relationship with an "Order" entity, indicating that one customer can place multiple orders. In object-oriented programming, relationships are represented through references or associations between classes, enabling entities to collaborate in modeling complex systems. (Orakzai, 2023).

Figure 67 Relationship between the users and transaction
Entity Relationship Diagram.

Figure 68 Entity relationship diagram.

Class and Code Structure.

Code Structure.

Code structure refers to the organization and layout of code in a software project. It encompasses how code is systematically arranged and modularized to improve readability, maintainability, and scalability. Proper code structure is crucial for allowing developers to understand the flow of the program, facilitating easier debugging, enhancing collaboration, and supporting the growth of the application with new features or during refactoring efforts.

Figure 69 Code structure.

API and External Interfaces.

This project incorporates a range of dependencies essential for enhancing functionality and user experience across multiple platforms. These packages support everything from user interface development and data handling to security management and document processing, ensuring a robust, scalable, and secure application.

IronPdf (2023.8.6)

Purpose: IronPdf is a library that allows developers to create, read, and edit PDF files programmatically in .NET. It can also convert HTML to PDF, which is useful for reports or web page receipts in applications.

Usage: This package might be used in your project to generate PDF documents from application data or directly from HTML sources.

Microsoft.AspNetCore.Component

Purpose: This package is part of the ASP.NET Core framework and provides support for building components in Blazor applications. These components are reusable UI elements that can encapsulate rendering logic and data.

Usage: It's likely used for constructing the interactive, client-side user interfaces in your web application.

Microsoft.AspNetCore.WebUtilities

Purpose: This package contains utilities to help with web-based operations, such as managing form data, encoding/decoding URLs, and file handling.

Usage: Useful in handling various web operations like parsing form data in requests or manipulating query strings.

Microsoft.Extensions.Logging.Debug

Purpose: Provides support for logging to the Visual Studio Debug window. It's part of the Microsoft logging extensions that allow for configurable logging in applications.

Usage: In your application, this would allow for logging important information during development, which can be viewed directly in the IDE's debug output window.

Microsoft.Maui.Controls (9.0.0)

Purpose: This is part of the .NET MAUI framework, which stands for Multi-platform App UI. It is used for building native mobile and desktop apps from a single shared codebase.

Usage: If your application targets multiple platforms (iOS, Android, macOS, Windows), this package provides the UI controls and layouts that will render natively on each platform.

System.IdentityModel.Tokens.Jwt

 24 Purpose: This package is used to create, validate, and decode JSON Web Tokens (JWT) in .NET. JWTs are an open standard used primarily for securely transmitting information between parties as a JSON object.

 15 Usage: Typically used in the authentication and authorization processes of your application, especially if using token-based authentication.

Conclusion

The comprehensive coursework embarked upon in 2024-25 has culminated in the successful development of "SpendWise," a desktop application tailored for efficient personal finance management. Utilizing the robust .Net Framework and the versatile C# programming language, the project has successfully integrated essential features for budget tracking, expense management, and financial planning.

The primary objective of "SpendWise" was to create a desktop application that simplifies personal finance management for users, enabling them to track their finances efficiently and make informed financial decisions.

The development of "SpendWise" revealed the effectiveness of C# in creating a scalable and secure application environment. Features such as transaction tracking, debt management, and financial reporting were integrated seamlessly, providing users with comprehensive tools to oversee their financial health.

The application's deployment demonstrates significant implications for personal financial management, suggesting that individuals can achieve greater financial insight and control through digital solutions. It validates the proposition that advanced technological tools can enhance personal financial planning and management.

For users, it is recommended to regularly utilize the reporting features of "SpendWise" to gain insights into their financial patterns and adjust their budgets accordingly.

Developers should consider continuous integration of feedback for feature enhancements to adapt to the evolving financial needs of users.

The current version of "SpendWise" primarily supports basic transaction management and simple financial reports. It may not cater to more complex financial needs such as investment tracking or multi-currency support, which could limit its applicability for more sophisticated financial users.

Future enhancements could include advanced analytics features, integration with banking APIs for real-time transaction data, and support for multiple currencies to cater to a global user base. Research could also explore the implementation of AI to provide predictive financial insights based on user spending habits.

"SpendWise" has successfully demonstrated how the .Net Framework and C# can be leveraged to develop robust personal finance applications. The experience and knowledge gained from developing "SpendWise" underscore the critical role of tailored financial tools in empowering individuals to manage their finances more effectively and with greater confidence. As technology evolves, the potential to further enhance such tools will undoubtedly play a pivotal role in the financial technology landscape.

Appendix

Opening Project and Starting Application.

Initially, the project solution file can be located by navigating to the project folder. After which, it should be opened with Microsoft Visual Studio 2022.

Figure 70 Opening Project in Visual Studio

Figure 71 Starting Application

The start button should be clicked from the IDE, which will launch the project's primary method and fire up the application. At first, the login page will be displayed.

Figure 72 Login Page

Testing

Go Back

Test case to verify username uniqueness..

Figure 73 Username uniqueness testing]

Password validation.

Figure 74 Password Validation

Test case to ensure debts are cleared correctly only from cash inflows.

Figure 75 Dashboard before clearing the debt\

Figure 76 Clearing debt btn

Figure 77 after clearing the pending debts

Figure 78 Dashboard after clearing the debts

Ensure top 5 highest/lowest transactions are displayed correctly.

Figure 79 Top 5 highest and lowest transaction

Figma Designs.

Register Design.

The Registration Page of the SpendWise project enables users to create an account by entering key details such as a username, account number, preferred currency, and password. Additionally, users are required to specify their chosen currency for budgeting purposes. This page ensures secure registration by implementing robust validation checks and establishes a solid foundation for tailored expense tracking.

Figure 80 Register page of my project SpendWise

Login design.

The Login Page provides a secure authentication process to safeguard users' sensitive financial information from unauthorized access. Upon successful authentication, users gain access to features such as expense tracking, budgeting tools, and debt management, ensuring a protected and efficient user experience.

Figure 81 Login page of my project SpendWise

Dashboard design.

The Dashboard functions as the primary hub of the SpendWise application, offering users a concise overview of their financial activities. It highlights essential statistics, including total inflows, outflows, debts, cleared debts, and outstanding debts. Users can also review pending debts, filter data based on specific date ranges, and analyze the top five highest or lowest transactions to enhance expense management effectively.

Figure 82 Dashboard of my project SpendWise

CashIn design.

The Cash In Page in the SpendWise project is designed to allow users to record and manage their income transactions efficiently. It focuses on capturing essential details of cash inflows to keep track of all income sources.

Figure 83 CashIn Flow page of my project SpenedWise

CashOut Flow design.

The Cash In Page within the SpendWise project is specifically designed to streamline the recording and management of income transactions. It emphasizes capturing critical details of cash inflows to effectively monitor and track all sources of income.

Figure 84 CashhOut Flow page of my project SpendWise

Transaction design

The Transactions page in the SpendWise application is meticulously designed to provide a comprehensive view of all financial transactions, both income and expenses. It allows for detailed oversight of transaction history by featuring filters such as Transaction Type, Tag, and date range selections. This interface ensures that users can easily categorize, search, and manage their financial data efficiently, enhancing the ability to track and analyze spending and income patterns over time.

Figure 85 Transaction page of my project SpendWise

Glossary of terms.

A Glossary of Terms in the context of software documentation or project description is a section where specific terminology used throughout the project or application is defined and explained. This is particularly important for ensuring that all stakeholders,

including developers, managers, and users, have a common understanding of the terms and concepts used within the project. Here's a detailed explanation of what might be included in a glossary for a financial or transaction-based application like SpendWise:

Term

Definition

Tags

Keywords or labels assigned to transactions to facilitate easier categorization and retrieval. Tags help users organize and identify transactions by common attributes such as "rent", "food", or "salary".

Debit

A transaction type where money is withdrawn from a user's account. Debits typically represent expenses or cash outflows.

Credit

A transaction type where money is added to a user's account. Credits generally represent income or cash inflows.

Debt

A financial obligation that a user owes to another entity. In the context of the application, it typically represents money that must be repaid to a creditor.

Date Range

A specific start and end date used to filter transactions over a set period. This helps users focus on entries relevant to a particular timeframe for better financial planning or analysis.

 PDF

Portable Document Format, a file format used to present and exchange documents reliably, independent of software, hardware, or operating system. In financial applications, PDFs may be used to export or share financial reports or statements.

JSON

 JavaScript Object Notation, a lightweight format for storing and transporting data, often used in web applications for server-client data exchanges. JSON is both easy for humans to read and write, and easy for machines to parse and generate.

Transaction

An action recorded in the financial system that involves the transfer of funds between accounts or parties. Each transaction records details such as the amount, date, and involved parties, and it can be classified as a debit, credit, or debt.

Table 17 Test Case 17

Detailed list of default tags and date ranges.

Default Tags

Default tags are predefined categories that users can apply to their transactions to facilitate easy sorting, filtering, and reporting. Here is a list of common default tags used in financial applications:

Utilities: For transactions related to utility payments such as electricity, water, gas, etc.

Groceries: For shopping related to food and household supplies.

Dining Out: For expenses incurred at restaurants and cafes.

Transportation: Includes expenses for public transport, fuel, car maintenance, etc.

Healthcare: Medical expenses, pharmacy costs, and health insurance.

Entertainment: Expenses on movies, events, hobbies, and leisure activities.

Rent/Mortgage: Payments towards housing or property loans.

Salary: Tag for income from salaries or wages.

Investments: Transactions related to investments like stocks, bonds, real estate, etc.

Taxes: For tracking tax payments.

Miscellaneous: For expenses or income that do not fit into other categories.

Default Date Ranges

Default date ranges are used to filter transaction data over specific periods, helping users review their financial activity over manageable intervals. Common default date ranges include:

Older: Transactions occurring 1st.

Newest: Transaction that occurs latently.

Custom Range: Allows users to specify a custom start and end date to filter transactions.